



BRAINWARE UNIVERSITY

BRAINWARE UNIVERSITY

PYTHON PROJECT-BASED LEARNING REPORT (WEEK 1)

1. Basic Information

Title of the Project:	Student Grade Management System (Terminal-Based)
Name(s) of Student(s):	Soumyadwip Das (BWU/BTS/25/169) Ankita Paul (BWU/BTS/25/180) Sonia Naskar (BWU/BTS/25/195) Joyeta Mondal (BWU/BTS/25/214) Sayantan Bharati (BWU/BTS/25/503)
Programme & Semester:	B.Tech CSE & 3rd Semester
Course Name & Code:	Python Programming Lab (BES09013)
Name of the Guide/Supervisor:	MR. PRANAB GHARAI & MR. INDRANIL DAS



2. Introduction and Background

Student grade management is an important academic task involving the recording, calculation, and retrieval of student performance. This project develops a console-based application that manages student records in memory using Python. Week 1 focuses on implementing the core features, including grade calculation, a menu-driven interface, and operations to add, view, search, update, and delete student records.

3. Objectives of the Project (Week 1)

- Develop a menu-driven application for managing student records.
- Implement Add, View, Search, Update, and Delete operations.
- Calculate grades from percentage scores.
- Build a reusable bordered terminal interface.
- Validate roll numbers and marks.
- Test all CRUD operations.

4. Problem Statement / Driving Question

How can student records, including marks, percentages, and grades, be efficiently managed using a simple terminal-based application?

5. Methodology (Week 1 Activities)

1. Data Model & Storage:

- Used an in-memory STUDENTS dictionary to store student records and a fixed SUBJECTS list for consistent data handling.

2. Grade Calculation Logic:

- Implemented `calculate_grade()` to assign letter grades and `get_report()` to calculate total marks, percentage, and grades

3. Terminal UI & Menu System:

- Developed reusable box-drawing functions for the interface, implemented a menu with Add, View, Search, Update, Delete, and Exit options, and added input validation for roll numbers and marks.

6. Expected Outcomes / Deliverables (Week 1)

- A working console application supporting Add, View, Search, Update, and Delete of student grade records
- A functioning `calculate_grade()` module producing consistent letter grades from percentage scores
- A bordered, readable terminal UI built from reusable box-drawing utilities
- Input validation for duplicate roll numbers, invalid marks, and non-numeric entries
- Week 1 project report



7. Core Logic

```
def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B+"
    elif percentage >= 60:
        return "B"
    elif percentage >= 50:
        return "C"
    elif percentage >= 40:
        return "D"
    else:
        return "F"

def get_report(roll):
    student = STUDENTS[roll]
    total = sum(student["marks"].values())
    max_total = 100 * len(SUBJECTS)
    percentage = (total / max_total) * 100
    grade = calculate_grade(percentage)
    return student, total, percentage, grade
```

calculate_grade() assigns a grade based on the percentage, while get_report() calculates the total, percentage, and final grade. Together, they provide the core logic for generating student reports across the application.



```

STUDENT
GRADE
MANAGEMENT
SYSTEM

Terminal Student Grade Management System

[1] Add Student
[2] View All Students
[3] Search Student
[4] Update Marks
[5] Delete Student
[6] Exit

> Enter option: 1
> Enter roll number: 002
> Enter student name: nam
> Enter marks in Python (out of 100): 55
> Enter marks in DBMS (out of 100): 55
> Enter marks in COA (out of 100): 55

STUDENT ADDED:

Roll no : 002
Name    : nam

[1] Add Student
[2] View All Students
[3] Search Student
[4] Update Marks
[5] Delete Student
[6] Exit

> Enter option: 
```

Page 4 | Student Grade Management System | Week 1 Report



9. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language
Python Dictionary (<code>dict</code>)	Store student records in memory
Python Standard I/O	Handle menu-driven user interaction
Exception Handling	Validate input and handle errors
Box-Drawing Characters	Create the bordered terminal interface

10. References

- Python Official Documentation
- Python Dictionaries Documentation
- Python Errors and Exceptions Documentation
- Unicode Box Drawing Characters Reference
- Brainware University – Python Programming Course Guidelines



BRAINWARE UNIVERSITY

11. Signatures

Signature of Student(s):

Date of Submission:

Signature of Guide/Supervisor:
